TP — Détection et correction d’une injection SQL avec SonarQube

Module : Pilotage & amélioration continue de la sécurité
Entreprise fil rouge : NOVA SERVICES
Application étudiée : TIWAP — Totally Insecure Web Application Project
Durée indicative : 2h à 3h
Technologies : Docker Compose, SonarQube Community Build, PostgreSQL, SonarScanner CLI, Python

1. Objectifs pédagogiques

NOVA SERVICES souhaite améliorer la sécurité de ses applications avant leur mise en production. L’équipe technique veut intégrer une analyse statique du code dans son processus de développement afin de repérer les défauts de sécurité plus tôt.

Dans ce TP, vous allez déployer SonarQube et PostgreSQL avec Docker Compose, analyser le code source de TIWAP, examiner les résultats et corriger une injection SQL dans un environnement pédagogique.

À la fin du TP, vous devrez savoir :

- Déployer une plateforme d’analyse de code avec Docker Compose.
- Configurer une analyse de code Python avec SonarScanner.
- Comprendre le principe d’une injection SQL.
- Identifier et qualifier une anomalie de sécurité signalée par l’analyse statique.
- Corriger une requête SQL vulnérable avec des paramètres.
- Relancer l’analyse et vérifier les résultats.
- Proposer des indicateurs de suivi pour le pilotage de la sécurité de NOVA SERVICES.

Point important : SonarQube ne garantit pas la détection de toutes les vulnérabilités. Les résultats dépendent notamment de la version, des règles activées et de la manière dont le code est écrit. L’objectif est donc de confronter les résultats de l’outil à une analyse technique, et non de considérer automatiquement tout rapport comme exhaustif.

C’est quoi une injection SQL ?

Une injection SQL est une vulnérabilité qui permet à un utilisateur de modifier le sens d’une requête SQL en envoyant une entrée spécialement conçue.

Elle apparaît généralement lorsque l’application insère directement les données saisies par l’utilisateur dans une requête SQL, sans utiliser de paramètres.

1. Exemple simple

Imaginons que NOVA SERVICES possède une application qui permet de rechercher un client par son nom.

Le code vulnérable construit la requête ainsi :

username = request.form["username"]query = f"SELECT * FROM users WHERE username = '{username}'"cursor.execute(query)

Si l'utilisateur saisit `alice`, la requête devient :

SELECT * FROM users WHERE username = 'alice';

Cela fonctionne normalement.

Mais si l'entrée utilisateur contient une expression SQL inattendue, elle peut modifier la logique de la requête. Selon le contexte, cela peut permettre de contourner un contrôle d'accès ou de consulter des données qui ne devraient pas être accessibles.

2. Comment corriger cette vulnérabilité ?

On utilise une requête paramétrée : la structure SQL est séparée des données fournies par l'utilisateur.

username = request.form["username"]query = "SELECT * FROM users WHERE username = ?"cursor.execute(query, (username,))

Ici, ? est le marqueur de paramètre utilisé par SQLite. Le marqueur exact dépend du pilote de base de données.

Même si l'utilisateur saisit une valeur inattendue, celle-ci est traitée comme une valeur, et non comme une partie de la commande SQL.

3. Pourquoi est-ce important pour NOVA SERVICES ?

Une injection SQL peut entraîner :

- Confidentialité : accès non autorisé à des données clients.
- Intégrité : modification ou suppression de données, selon les privilèges disponibles.
- Disponibilité : perturbation de l'application ou de la base de données.

2. Contexte : NOVA SERVICES

NOVA SERVICES développe et exploite plusieurs applications internes : gestion de clients, suivi des prestations et outils d'administration.

L'équipe sécurité souhaite intégrer une étape d'analyse statique au cycle de développement. Elle veut notamment :

- Repérer les défauts de sécurité avant la livraison.
- Prioriser les corrections selon leur gravité et leur exposition.
- Vérifier que les corrections apportées réduisent le risque.
- Suivre l'évolution des anomalies au fil des versions.

Pour apprendre à utiliser l'outil, vous étudierez **TIWAP**, une application de laboratoire pédagogique écrite avec Python et Flask, qui contient volontairement plusieurs vulnérabilités web, dont des injections SQL. Elle sert uniquement d'exemple d'analyse : elle n'est pas présentée comme une application de production de NOVA SERVICES.

Qu'est-ce que le SAST dans SonarQube ?

SAST signifie Static Application Security Testing, c'est-à-dire test statique de sécurité des applications.

C'est une méthode qui consiste à analyser le code source d'une application sans avoir besoin de l'exécuter, afin d'identifier des vulnérabilités potentielles et des mauvaises pratiques de programmation.

Dans notre TP NOVA SERVICES, nous utilisons SonarQube pour analyser le code Python de TIWAP et rechercher notamment des problèmes liés aux injections SQL.

https://www.trendmicro.com/fr_fr/what-is/proactive-security/sast-static-application-security-testing.html

https://docs.sonarsource.com/

https://docs.gitlab.com/user/application_security/sast/

Scénario de travail

Une analyse de code est demandée à la suite d'un audit interne. L'équipe souhaite vérifier si une requête SQL construite à partir d'une entrée utilisateur permet potentiellement de contourner les contrôles applicatifs.

Vous devrez produire un état initial, corriger une anomalie dans une copie de travail, puis comparer les résultats avant et après correction.

3. Prérequis

- Docker Engine et Docker Compose v2.
- Python 3.x et Git.
- Un navigateur web.
- Un éditeur de code, par exemple VS Code.

- Environ 4 Go de mémoire disponibles pour le laboratoire, idéalement davantage si votre machine le permet.

Vérifiez les outils :

```
docker --version
docker compose version
git --version
python3 --version
```

Attention: Sur un hôte Linux, SonarQube peut nécessiter des paramètres noyau adaptés à Elasticsearch. Si le démarrage échoue avec une erreur liée à **`vm.max_map_count`**, vérifiez la documentation correspondant à votre système et appliquez, si nécessaire :

```
sudo sysctl -w vm.max_map_count=524288
```

Cette modification est temporaire jusqu'au redémarrage, sauf si elle est configurée de manière persistante.

4. Préparer le laboratoire

Créez un dossier séparé pour le TP :

```
mkdir tp-sonarqube-nova
cd tp-sonarqube-nova
```

Clonez ensuite TIWAP dans ce dossier :

```
git clone https://github.com/tombstoneghost/TIWAP.git
```

La structure doit ressembler à ceci :

```
tp-sonarqube-nova/
├── TIWAP/
│   ├── ...
│   └── ...
└── compose.yaml
```

Nous allons **analyser les fichiers de TIWAP sans lancer l'application vulnérable**. Le fichier Compose de SonarQube sera séparé de celui de TIWAP afin de ne pas démarrer accidentellement les services de l'application.

Avant de continuer, vérifiez que le dépôt a bien été cloné :

```
ls
ls TIWAP
```

5. Déployer SonarQube et PostgreSQL

5.1. Créer le fichier Compose

👉Créez un fichier `compose.yaml` contenant :

Comprendre l'architecture :

- **db** héberge la base PostgreSQL utilisée par SonarQube pour stocker ses données.
- **`sonarqube`** expose l'interface web sur le port local 9000.
- **`scanner`** analyse les fichiers de TIWAP montés dans `/usr/src`.
- **`sonarnet`** permet aux conteneurs de communiquer entre eux par leur nom de service.
- Les volumes conservent les données de SonarQube et PostgreSQL entre les redémarrages.

Le port de l'interface est lié à `127.0.0.1` : l'interface n'est donc pas publiée sur toutes les interfaces réseau de l'hôte.

5.2. Configurer le mot de passe de la base

Créez un fichier `.env` dans le même dossier que `compose.yaml` :

POSTGRES_PASSWORD=PasseAChanger_123!

Ce mot de passe est un exemple de laboratoire, pas un secret adapté à la production.

Créez également un fichier `.gitignore` :

.env

Ne versionnez jamais les secrets ou les jetons d'analyse. Dans un environnement réel, utilisez un gestionnaire de secrets et des identifiants adaptés.

5.3. Démarrer les services

Lancez les conteneurs :

docker compose up -d

Vérifiez leur état :

docker compose ps

Consultez les journaux si nécessaire :

docker compose logs --tail=100 db
docker compose logs --tail=100 sonarqube

SonarQube peut mettre plusieurs minutes à démarrer. Vérifiez son état depuis votre machine :

curl -s http://localhost:9000/api/system/status

Attendez que le statut indique que le service est opérationnel. Si `curl` n'est pas installé, ouvrez directement l'interface dans le navigateur.

Accédez à :

`http://localhost:9000`

Lors d'une première installation, les identifiants initiaux sont généralement **`admin` / `admin`**. Suivez la procédure de changement de mot de passe demandée par l'interface.

Question 1 : Pourquoi SonarQube utilise-t-il PostgreSQL plutôt que de stocker toutes ses données dans le conteneur applicatif ?

Question 2 : Pourquoi l'adresse **`http://sonarqube:9000`** est-elle utilisée entre conteneurs, alors que le navigateur utilise `http://localhost:9000` ?

6. Créer un jeton d'analyse

SonarScanner doit s'authentifier auprès de SonarQube pour envoyer les résultats de son analyse.

Dans l'interface SonarQube :

1. Connectez-vous.
2. Ouvrez les paramètres du compte, puis la section **Security** ou **Sécurité**.
3. Générez un jeton dédié à l'analyse.
4. Copiez-le immédiatement et conservez-le temporairement dans votre terminal.

Les libellés exacts peuvent varier selon la version.

Dans le terminal, depuis le dossier du TP :

export SONAR_TOKEN='COLLER_ICI_LE_JETON'

Sous PowerShell, utilisez :

$env:SONAR_TOKEN = "COLLER_ICI_LE_JETON"

Ne mettez pas le jeton dans **`sonar-project.properties`**, dans le dépôt Git ou dans une capture d'écran. Ne le partagez pas dans un rapport.

Pour un usage réel, privilégiez un jeton limité au projet et révoquez-le lorsqu'il n'est plus nécessaire.

7. Préparer la configuration de l'analyse

À la racine du dépôt `TIWAP`, créez un fichier `sonar-project.properties` :

```
sonar.projectKey=tiwap
sonar.projectName=TIWAP
sonar.projectVersion=1.0
sonar.sources=.
sonar.sourceEncoding=UTF-8
```

Le fichier doit être placé ici :

```
tp-sonarqube-nova/
└── TIWAP/
    ├── sonar-project.properties
    └── ...
```

Le scanner utilisera la configuration du projet et les variables d'environnement **`SONAR_HOST_URL`** et **`SONAR_TOKEN`**.

Remarque : pour ce TP, nous analysons principalement les fichiers source. Une configuration plus précise pourra exclure les environnements virtuels, caches, fichiers générés et autres répertoires non pertinents s'ils sont présents.

8. Réaliser l'analyse initiale

Depuis le dossier `tp-sonarqube-nova`, vérifiez que le jeton est défini :

```
test -n "$SONAR_TOKEN" && echo "Jeton défini"
```

Sous PowerShell :

```
if ($env:SONAR_TOKEN) { "Jeton défini" }
```

Lancez le scanner :

```
docker compose run --rm \
  -e SONAR_TOKEN="$SONAR_TOKEN" \
  scanner
```

Sous PowerShell, vous pouvez utiliser :

```
docker compose run --rm -e SONAR_TOKEN=$env:SONAR_TOKEN scanner
```

Le scanner s'exécute dans le réseau Docker. Il contacte donc SonarQube via `http://sonarqube:9000`, et non via `localhost:9000`.

À la fin, consultez le tableau de bord :

`http://localhost:9000`

Ouvrez le projet `tiwap` et examinez les résultats.

8.1. Examiner les résultats

Recherchez notamment :

- Les vulnérabilités identifiées.
- Les Security Hotspots, s'ils sont présentés.
- Les problèmes de fiabilité ou de maintenabilité.
- Les fichiers et lignes concernés.
- La règle associée à chaque résultat.
- Les recommandations de correction fournies par l'outil.

Pour chaque résultat intéressant, consignez :

Élément	**Informations à relever**
Identifiant	Référence de l'anomalie dans SonarQube
Fichier et ligne	Emplacement dans le code
Règle	Règle déclenchée par l'analyse
Type	Vulnérabilité, hotspot ou autre problème
Impact potentiel	Conséquence possible sur l'application
Confiance	Le résultat est-il confirmé par l'examen du code ?
Action	Corriger, examiner davantage ou documenter

Ne concluez pas qu'une vulnérabilité est absente uniquement parce que le scanner ne la signale pas. Une analyse statique peut manquer certains cas ou nécessiter des règles complémentaires.

9. Comprendre l'injection SQL

Une injection SQL se produit lorsqu'une application construit une requête SQL en intégrant directement des données contrôlées par un utilisateur.

Considérons cet exemple pédagogique :

```
username = request.form["username"]

query = (
   "SELECT id, username FROM users "
   f"WHERE username = '{username}'"
)

cursor.execute(query)
```

Le problème est que la donnée utilisateur est incorporée dans le texte de la requête. Elle peut alors modifier la structure SQL attendue.

Le défaut n'est pas simplement l'utilisation d'une chaîne de caractères : c'est l'absence de séparation fiable entre **la requête SQL** et **les valeurs transmises à la base**.

9.1. Retrouver le code concerné dans TIWAP

Dans un terminal, depuis `tp-sonarqube-nova`, recherchez les requêtes SQL et les points d'entrée utilisateur :

```
grep -RInE \
  "execute\(|SELECT|request\.args|request\.form" \
  TIWAP \
  --include="*.py"
```

Cette commande fournit des pistes de recherche ; elle ne prouve pas qu'une ligne est vulnérable.

Vous pouvez également rechercher dans le code les noms de variables qui contiennent les entrées utilisateur et remonter jusqu'à la requête exécutée.

Pour le résultat sélectionné dans SonarQube :

1. Ouvrez le fichier concerné.
2. Identifiez l'entrée utilisateur.
3. Suivez son utilisation jusqu'à la requête SQL.
4. Vérifiez si les valeurs sont concaténées ou interpolées dans le SQL.
5. Notez les éléments permettant de confirmer ou d'écarter le résultat.

TIWAP contient plusieurs catégories de vulnérabilités. Ne supposez pas que toutes les requêtes SQL du projet sont vulnérables : examinez précisément le chemin de données associé à l'anomalie.

10. Corriger la requête SQL (optionnel)

La correction consiste à utiliser des requêtes paramétrées, également appelées *requêtes préparées*.

10.1. Exemple de correction avec SQLite

Code vulnérable :

```
username = request.form["username"]

query = (
   "SELECT id, username FROM users "
   f"WHERE username = '{username}'"
)

cursor.execute(query)
```

Code corrigé :

```
username = request.form["username"]

query = (
   "SELECT id, username FROM users "
   "WHERE username = ?"
)

cursor.execute(query, (username,))
```

Le marqueur ? est adapté à l'interface SQLite utilisée dans cet exemple. D'autres pilotes SQL utilisent des marqueurs différents : il faut respecter la syntaxe attendue par le pilote concerné.

Le principe à retenir est le suivant : la structure SQL est définie séparément des données. Le pilote transmet les paramètres comme des valeurs, plutôt que de les incorporer au texte SQL.

Attention : ce code illustre une correction générique. Adaptez-la à la fonction réelle de TIWAP, à son pilote de base de données et aux types de retour attendus. Ne remplacez pas aveuglément une requête complète sans vérifier son contexte.

10.2. Bonnes pratiques complémentaires

- Utiliser des requêtes paramétrées pour les valeurs.
- Valider les entrées selon le besoin métier.
- Utiliser des comptes de base de données aux privilèges minimaux.
- Ne pas considérer une validation de chaîne ou une suppression de caractères comme un remplacement des requêtes paramétrées.
- Éviter de retourner aux utilisateurs les messages SQL détaillés.
- Ajouter des tests de non-régression pour les chemins concernés.

Si le code permet de choisir dynamiquement un nom de table ou une colonne, les paramètres SQL classiques ne remplacent pas une liste blanche de valeurs autorisées pour ces identifiants.

11. Mettre en place une copie de correction

Pour éviter de modifier le dépôt original sans contrôle, créez une branche de travail ou une copie locale dédiée.

Depuis `TIWAP` :

```
cd TIWAP
git status
git switch -c fix/sql-injection-lab
```

Si le dépôt ne permet pas de créer une branche ou si vous préférez ne pas le modifier directement, réalisez une copie locale et travaillez dessus.

Modifiez ensuite uniquement la fonction concernée, en conservant le comportement fonctionnel attendu.

Avant de relancer l'analyse, vérifiez les changements :

```
git diff
```

Puis exécutez les tests disponibles dans le dépôt. Si aucun test ne couvre la fonction corrigée, proposez un test de régression adapté et expliquez comment il vérifie le comportement attendu.

N'exécutez pas l'application vulnérable sur un serveur public. Le laboratoire doit rester isolé.

12. Relancer l'analyse et comparer les résultats

Depuis le dossier `tp-sonarqube-nova`, relancez :

```
docker compose run --rm \
  -e SONAR_TOKEN="$SONAR_TOKEN" \
  scanner
```

Retournez dans SonarQube et consultez la nouvelle analyse.

Comparez l'état initial et l'état après correction :

Critère	Avant correction	Après correction
Anomalie étudiée	À compléter	À compléter

Règle concernée	À compléter	À compléter
Emplacement	À compléter	À compléter
Code vulnérable présent ?	À vérifier	À vérifier
Tests de régression	À compléter	À compléter
Risque résiduel	À qualifier	À qualifier

L'anomalie peut ne plus apparaître, changer de statut ou rester visible selon la règle et la version de l'analyseur. Dans tous les cas, justifiez votre conclusion à partir du code, du rapport et des tests.

Si le résultat ne change pas :

1. Vérifiez que vous avez modifié le fichier réellement analysé.
2. Vérifiez que le scanner utilise bien le répertoire `TIWAP`.
3. Examinez la règle et le type de résultat signalé.
4. Vérifiez que le résultat n'est pas un Security Hotspot qui nécessite une revue manuelle.
5. Consultez les journaux du scanner.

Un résultat inchangé ne signifie pas automatiquement que la correction est mauvaise ; il faut d'abord comprendre ce que la règle mesure.

13. Relier le TP au pilotage de la sécurité de NOVA SERVICES

SonarQube est un moyen technique. Le pilotage de la sécurité consiste à exploiter ses résultats pour décider, suivre les corrections et vérifier leur efficacité.

Pour NOVA SERVICES, chaque anomalie retenue devrait être associée à :

- Un actif ou une application concernée.
- Un responsable de correction.
- Une priorité fondée sur le risque métier.
- Une échéance.
- Un statut de traitement.
- Une preuve de correction et de vérification.
- Un risque résiduel, si nécessaire.

13.1. Prioriser une anomalie

Une injection SQL sur une application exposée à Internet qui traite des données clients ne présente pas nécessairement le même risque qu'une anomalie sur un outil de test isolé.

Pour proposer une priorité, prenez en compte :

1. La criticité de l'application et des données.
2. L'exposition de l'application.
3. L'exploitabilité du défaut.
4. Les conséquences possibles sur la confidentialité, l'intégrité et la disponibilité.
5. Les mesures compensatoires déjà en place.

Une note de gravité fournie par un outil peut aider, mais ne remplace pas l'évaluation du risque métier.

13.2. Proposer des indicateurs

Pour le tableau de bord de NOVA SERVICES, proposez au moins trois indicateurs parmi les suivants :

- Nombre de vulnérabilités ouvertes, par niveau de priorité.
- Nombre de vulnérabilités critiques ou élevées dépassant leur échéance.
- Délai médian entre la détection et la correction.
- Pourcentage de corrections vérifiées par une nouvelle analyse.
- Taux de réouverture des anomalies précédemment corrigées.
- Part des projets applicatifs analysés dans le périmètre défini.

Pour chaque indicateur, précisez sa définition, sa source de données, sa fréquence de mise à jour, son responsable et la décision qu'il doit permettre de prendre.

Un indicateur n'est utile que s'il aide à décider et à agir. Le nombre brut d'anomalies ne suffit pas à mesurer, à lui seul, le niveau de sécurité d'une organisation.

14. Travail demandé

Réalisez les activités suivantes :

1. Déployer SonarQube et PostgreSQL avec Docker Compose.
2. Vérifier le démarrage des services et documenter l'architecture.
3. Créer un jeton d'analyse et lancer le scanner dans Docker.
4. Réaliser un état initial des résultats de TIWAP.
5. Identifier une anomalie liée à une injection SQL, si elle est signalée, et examiner le code associé.
6. Expliquer le mécanisme de la vulnérabilité à partir du code.
7. Corriger la requête avec des paramètres adaptés au pilote.
8. Exécuter les tests disponibles ou proposer un test de non-régression.
9. Relancer l'analyse et comparer les résultats.
10. Proposer une priorité, un responsable, une échéance et une preuve de correction dans le contexte de NOVA SERVICES.
11. Définir au moins quatre indicateurs de suivi.

Si SonarQube ne signale pas l'injection SQL ciblée, vous devez le mentionner dans votre compte rendu et documenter l'examen manuel du code. N'inventez pas un résultat de scanner.

15. Livrables attendus

Remettez un dossier contenant :

- Le fichier `compose.yaml`, sans secret.
- Le fichier `sonar-project.properties`.
- Une capture de l'interface SonarQube montrant le projet analysé.
- Un tableau d'analyse initiale des anomalies pertinentes.
- L'extrait du code vulnérable et sa version corrigée.
- Une explication technique de la correction.
- La preuve de l'exécution des tests ou le test de non-régression proposé.
- Une comparaison des résultats avant/après.
- Une fiche de traitement de l'anomalie dans le contexte de NOVA SERVICES.
- Un mini-tableau de bord avec au moins quatre indicateurs.
- Une courte conclusion sur les limites de l'analyse statique.

Aucun jeton, mot de passe ou autre secret ne doit apparaître dans les livrables.

16. Questions de compréhension

1. Quel est le rôle de PostgreSQL dans cette architecture ?
2. Pourquoi `localhost:9000` fonctionne-t-il depuis le navigateur mais n'est-il pas l'adresse à utiliser depuis le conteneur du scanner ?
3. Quelle différence existe-t-il entre une analyse statique et un test d'intrusion dynamique ?
4. Pourquoi la concaténation d'une entrée utilisateur dans une requête SQL crée-t-elle un risque ?
5. En quoi une requête paramétrée réduit-elle ce risque ?
6. Pourquoi une validation de l'entrée ne remplace-t-elle pas les requêtes paramétrées ?
7. Pourquoi un résultat de SonarQube doit-il être vérifié et qualifié ?
8. Que faire si aucune vulnérabilité SQL n'est signalée ?
9. Comment prioriser une vulnérabilité affectant une application métier critique ?
10. Pourquoi le nombre total d'anomalies n'est-il pas un indicateur suffisant du niveau de sécurité ?
11. Quelles protections faut-il ajouter autour du scanner et de son jeton dans un pipeline CI/CD ?
12. Comment prouver qu'une correction est effective et durable ?

17. Nettoyage du laboratoire

Lorsque vous avez terminé, arrêtez les conteneurs : docker compose down

Cette commande conserve les volumes. Pour supprimer également les données du laboratoire, vous pouvez utiliser : docker compose down -v

Attention : l'option `-v` supprime les volumes de données associés à ce projet. Ne l'utilisez pas si vous souhaitez conserver l'historique SonarQube ou les données PostgreSQL.

Si le jeton n'est plus utile, révoquez-le dans SonarQube.

18. Pour aller plus loin

- Ajouter l'analyse SonarQube à un pipeline GitLab CI ou GitHub Actions.
- Définir une politique de traitement des vulnérabilités avec des délais selon la priorité.
- Empêcher la fusion de code si des critères de qualité définis ne sont pas respectés.
- Ajouter des tests de sécurité et des tests de non-régression.
- Construire une vue de pilotage consolidant les résultats des analyses, les échéances et les corrections.

L'objectif final pour NOVA SERVICES n'est pas simplement d'installer SonarQube : il est de transformer les résultats d'analyse en décisions, en corrections vérifiables et en amélioration continue.